



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTE)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH, Hyderabad



III Year I Semester
KR24
ARTIFICIAL INTELLIGENCE
Department of CSE

UNIT-I PART-2 NOTES

Unit-I:

Introduction to Deep Learning: Fundamentals of Neural Networks and Deep Learning, Deep Learning Applications, Deep Learning Framework

Regression: Neuron Model, Optimization Method, Linear Model in Action.

Classification: Hand Written Digital Picture Dataset, Build a model, Error Calculation, Non-Linear model, model complexity, Optimization Method, Hands-On Hand-Written Digital Image Recognition.

Prepared by

M. ASHA

Assistant Professor

Department of CSE (AI & ML)

1. REGRESSION

1.1 Introduction to Regression

Regression is a supervised learning technique used to predict a continuous numerical output from one or more input variables.

Examples:

- Predicting house price
- Predicting temperature
- Predicting sales
- Predicting salary

In regression, the target variable is numerical and can take a range of values.

Example

Suppose we want to predict the price of a house based on its area:

Area (sq.ft)	Actual Price (₹ lakhs)
1000	45
1200	52
1500	65
1800	78
2000	85

The model learns the relationship:

Area → Price

and predicts the price of a previously unseen house.

1.2 Neuron Model

The basic building block of a neural network is an artificial neuron.

A neuron receives several inputs, multiplies each input by a corresponding weight, adds a bias, and applies an activation function.

The mathematical model is:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

or, in vector form:

$$z = w^T x + b$$

The output is:

$$\hat{y} = f(z)$$

where:

- $x_1, x_2, \dots, x_n$ = input features
- $w_1, w_2, \dots, w_n$ = weights
- b = bias
- z = weighted sum
- f = activation function
- $\hat{y}$ = predicted output

1.3 Single-Neuron Regression Model

For a single input x , the neuron becomes:

$$\hat{y} = wx + b$$

This is exactly the equation of a straight line:

$$y = mx + c$$

where:

$$w = m$$

and

$$b = c$$

Therefore, a single neuron with a linear activation can implement **linear regression**.

Example Suppose:

$$w = 3, b = 2$$

For:

$$x = 5$$

the prediction is:

$$\hat{y} = (3)(5) + 2$$

$$\hat{y} = 17$$

1.4 Role of Weights

A **weight** determines how strongly an input influences the output.

Consider:

$$\hat{y} = w_1x_1 + w_2x_2 + b$$

If:

$$w_1 = 5$$

and

$$w_2 = 0.2$$

then x_1 has a much stronger contribution to the prediction than x_2 , assuming comparable input scales.

During training, the network learns appropriate values of the weights.

1.5 Role of Bias

Bias allows the model to shift its output.

Without bias:

$$\hat{y} = wx$$

With bias:

$$\hat{y} = wx + b$$

For example:

$$y = 2x + 5$$

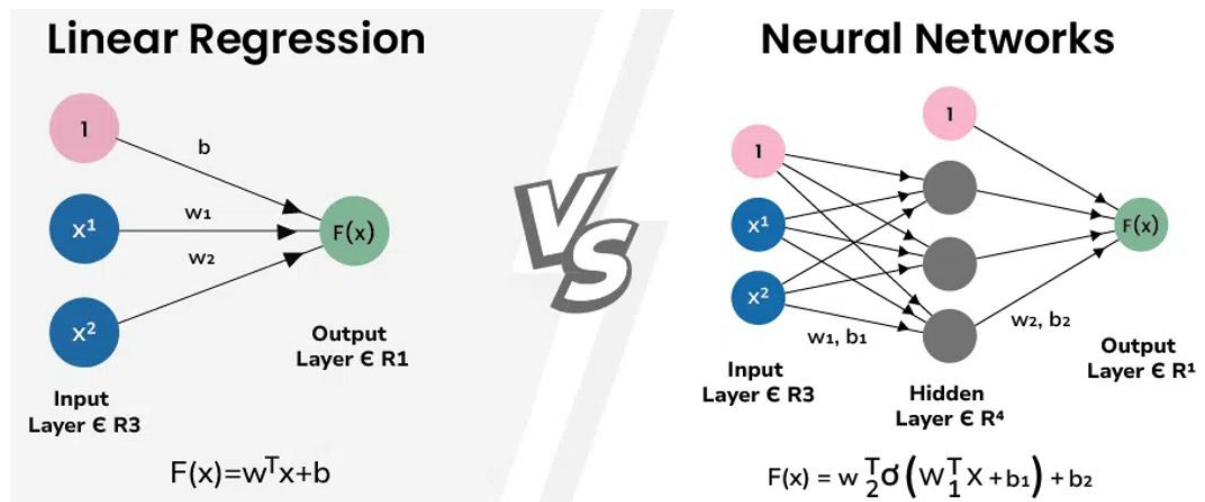
Here:

- Weight/slope = 2
- Bias/intercept = 5

Bias provides additional flexibility to the model.

1.6 Linear Regression Using a Neural Network

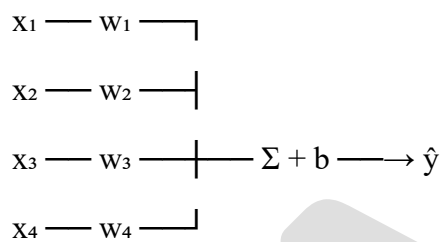
A single-neuron linear regression model can be represented as:



Mathematically:

$$\hat{y} = wx + b$$

For multiple features:



$$\hat{y} = w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + b$$

1.7 Error in Regression

After producing a prediction, we compare it with the actual value.

$$Error = y - \hat{y}$$

where:

- y = actual value
- $\hat{y}$ = predicted value

Example

Actual:

$$y = 20$$

Predicted:

$$\hat{y} = 17$$

Therefore:

$$Error = 20 - 17 = 3$$

1.8 Mean Squared Error

One of the most commonly used regression loss functions is **Mean Squared Error (MSE)**.

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Example

Suppose:

Actual	Predicted	Error	Squared Error
10	8	2	4
20	22	-2	4
30	29	1	1

Therefore:

$$MSE = \frac{4 + 4 + 1}{3}$$

$$MSE = 3$$

1.9 Why Square the Error?

If we simply calculate:

$$\frac{1}{N} \sum (y - \hat{y})$$

positive and negative errors may cancel each other.

For example:

$$2 + (-2) = 0$$

even though both predictions have errors.

Squaring gives:

$$2^2 + (-2)^2 = 4 + 4 = 8$$

Thus, MSE:

- Makes all errors positive
- Penalizes large errors more strongly
- Provides a differentiable objective for optimization

1.10 Optimization Method

The objective of training is to find weights and bias that minimize the loss.

For a linear regression model:

$$\hat{y} = wx + b$$

the loss can be written as:

$$L(w, b) = \frac{1}{N} \sum_{i=1}^N (y_i - (wx_i + b))^2$$

We want to find:

$$w^*, b^*$$

such that:

$$L(w^*, b^*)$$

is minimum.

This is an **optimization problem**.

1.11 Gradient Descent

Gradient descent is an optimization algorithm used to minimize the loss function.

The basic update rule is:

$$\theta_{new} = \theta_{old} - \eta \frac{\partial L}{\partial \theta}$$

where:

- θ = parameter
- L = loss
- η = learning rate
- $\frac{\partial L}{\partial \theta}$ = gradient

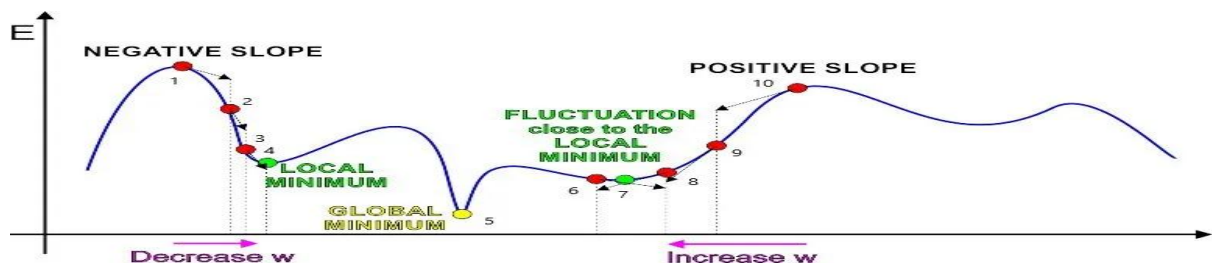
For the weight:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

For the bias:

$$b_{new} = b_{old} - \eta \frac{\partial L}{\partial b}$$

1.12 Intuition of Gradient Descent



The gradient indicates the direction in which loss increases.

Therefore, we move in the **opposite direction of the gradient**.

1.13 Learning Rate

The learning rate η controls the size of each update.



Very small learning rate

Small updates



Slow training

Very large learning rate

Large updates



May overshoot minimum



Training may become unstable

Appropriate learning rate

Controlled updates



Faster and stable convergence

1.14 Optimization Example

Suppose:

$$w = 2$$

and the gradient is:

$$\frac{\partial L}{\partial w} = 4$$

Let:

$$\eta = 0.1$$

Then:

$$w_{new} = 2 - (0.1)(4)$$

$$w_{new} = 1.6$$

The weight moves in the direction that is expected to reduce the loss.

1.15 Linear Model in Action Using Keras

A single-neuron regression model can be created using Keras:

```
import keras  
  
from keras import layers  
  
model = keras.Sequential([  
    keras.Input(shape=(1,)),  
    layers.Dense(1)])
```

The model represents: $\hat{y} = wx + b$

Compile the model:

```
model.compile(optimizer="sgd",loss="mse")
```

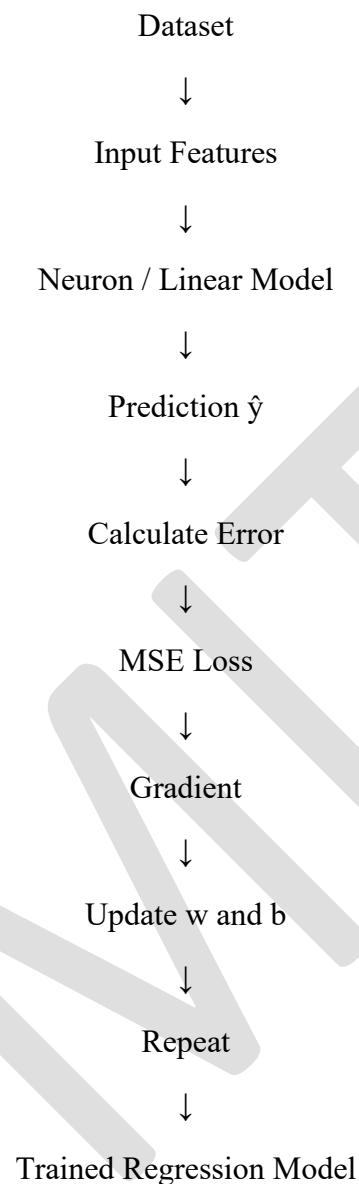
Train:

```
history = model.fit(x_train,y_train,epochs=100,batch_size=32)
```

Predict:

```
predictions = model.predict(x_test)
```

1.16 Regression Workflow



2. CLASSIFICATION

2.1 Introduction to Classification

Classification is a supervised learning problem in which the model predicts a discrete class or category.

Examples:

- Cat vs Dog
- Spam vs Not Spam
- Digit 0–9
- Benign vs Malignant

Unlike regression, classification does not generally predict a continuous numerical value as its final output.

2.2 Types of Classification

Binary Classification: Two classes.

Example:

0 → Not Spam

1 → Spam

Typical output activation:

Sigmoid

Multiclass Classification: More than two classes.

Example:

0

1

2

3

...

9

MNIST is a multiclass classification problem with 10 classes.

Typical output activation:

Softmax

Multilabel Classification

An input can belong to multiple classes simultaneously.

For example, an image may contain:

Car + Person + Road

This is different from multiclass classification, where normally one class is selected from mutually exclusive alternatives.

2.3 Handwritten Digital Picture Dataset — MNIST

The **MNIST dataset** is a standard dataset used to demonstrate handwritten digit recognition.

It contains images of handwritten digits:

0,1,2, ...,9

Therefore:

10 classes

Each image is:

28×28

pixels and is grayscale.

Therefore, the number of pixels is:

$$28 \times 28 = 784$$

2.4 MNIST Data Representation

An individual image can be represented as:

28×28

matrix.

Each value represents pixel intensity.

For a grayscale image:

0 = black

and approximately:

255 = white

depending on the image representation convention.

2.5 Loading MNIST

Keras provides MNIST directly:

```
from keras.datasets import mnist  
  
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

The training and test data contain:

x_train → Images

y_train → Labels

x_test → Images

y_test → Labels

2.6 Normalization of MNIST

Pixel values are generally represented in the range: 0 to 255

We can normalize them:

```
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

Now the values are approximately:

0 to 1

Why normalize?

Normalization can:

- Improve numerical stability
- Make optimization easier
- Help the model converge more efficiently

2.7 Building a Classification Model

A basic neural network for MNIST:

```
import keras
from keras import layers
model = keras.Sequential([
    keras.Input(shape=(28, 28)),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(10, activation="softmax")])
```

Architecture:

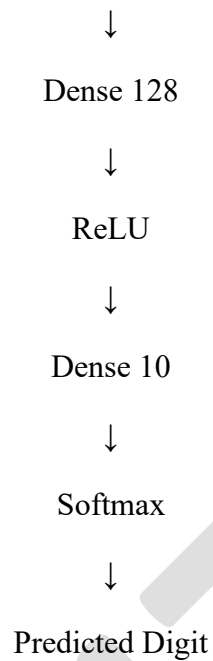
28×28 Image



Flatten



784



2.8 Flatten Layer

The original image has:

$$28 \times 28$$

dimensions.

The Dense layer expects a vector, so we flatten the image:

$$28 \times 28 = 784$$

In Keras:

```
layers.Flatten()
```

2.9 Hidden Layer

We use:

```
layers.Dense(128,activation="relu")
```

This creates 128 neurons.

The ReLU function is:

$$ReLU(x) = \max(0, x)$$

It introduces nonlinearity into the model.

2.10 Output Layer

MNIST contains 10 classes.

Therefore:

layers.Dense(10,activation="softmax")

There are 10 output neurons:

Neuron 0 → probability of digit 0

Neuron 1 → probability of digit 1

Neuron 2 → probability of digit 2

...

Neuron 9 → probability of digit 9

2.11 Softmax Activation

For K classes:

$$P_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Softmax converts the output scores into probabilities.

For example:

Digit 0 → 0.01

Digit 1 → 0.02

Digit 2 → 0.05

Digit 3 → 0.80

Digit 4 → 0.03

...

The probabilities sum approximately to:

1

The predicted class is the class having the highest probability.

$$\text{Predicted Class} = \arg \max_i P_i$$

2.12 Compile the Classification Model

For MNIST, labels are integer values:

0, 1, 2, ..., 9

Therefore, we can use sparse categorical cross-entropy:

```
model.compile(  
    optimizer="adam",  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"])
```

Three important components are specified:

Optimizer

optimizer="adam"

Controls parameter updates.

Loss

loss="sparse_categorical_crossentropy"

Measures classification error.

Metric

metrics=["accuracy"]

Measures classification accuracy.

2.13 Cross-Entropy Error

For multiclass classification:

$$L = - \sum_{i=1}^K y_i \log(\hat{y}_i)$$

where:

- y_i = actual target
- $\hat{y}_i$ = predicted probability

For a one-hot encoded target, only the correct class contributes to the loss:

$$L = -\log(\hat{y}_{correct})$$

Example

Suppose the actual digit is 3.

If:

$$P(3) = 0.9$$

then:

$$L = -\log(0.9)$$

$$L \approx 0.105$$

If:

$$P(3) = 0.1$$

then:

$$L = -\log(0.1)$$

$$L \approx 2.303$$

Therefore:

A model receives a much smaller loss when it assigns high probability to the correct class.

2.14 Accuracy

Accuracy is:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

Suppose the model correctly identifies:

950

out of:

1000

images.

Then:

$$\text{Accuracy} = \frac{950}{1000}$$

$$\text{Accuracy} = 95\%$$

2.15 Training the MNIST Model

```
history = model.fit(x_train,y_train,epochs=10,batch_size=32,validation_split=0.1)
```

Important parameters

Epochs Number of times the training data is passed through the model.

batch_size Number of samples processed before an optimization update.

validation_split Fraction of training data used for validation.

2.16 Testing the Model

After training:

```
test_loss, test_accuracy = model.evaluate(x_test,y_test)

print("Test Accuracy:", test_accuracy)
```

The test set provides an estimate of performance on unseen data.

2.17 Prediction

```
predictions = model.predict(x_test)

For the first image:

predicted_class = predictions[0].argmax()

print("Predicted:", predicted_class)

print("Actual:", y_test[0])
```

2.18 Non-Linear Model Complexity

A very important concept in classification is **model complexity**.

A simple linear model can create only a linear decision boundary.

For two features:

$$w_1x_1 + w_2x_2 + b = 0$$

represents a straight-line boundary.

But many real-world datasets require nonlinear boundaries.

2.19 Why Linear Models Are Limited

Consider the XOR problem:

x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	0

A single linear neuron cannot correctly separate these points.

Therefore, we need a nonlinear model.

2.20 Nonlinear Neural Network

A nonlinear neural network can be represented as:

$$h = f(W_1x + b_1)$$

followed by:

$$\hat{y} = W_2h + b_2$$

where f may be ReLU.

2.21 Increasing Model Complexity

Model complexity can be increased by:

- Increasing number of neurons
- Increasing number of hidden layers
- Adding nonlinear transformations
- Using more sophisticated architectures

Example:

Simple

Input $\rightarrow$ Output

More complex

Input

↓

Dense 128

↓

ReLU

↓

Dense 64

↓

ReLU

↓

Dense 10

↓

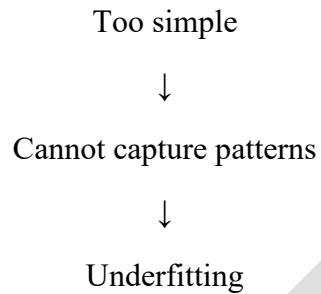
Softmax

The second model has greater representational capacity.

2.22 Model Capacity

Model capacity refers to the ability of a model to learn complex functions.

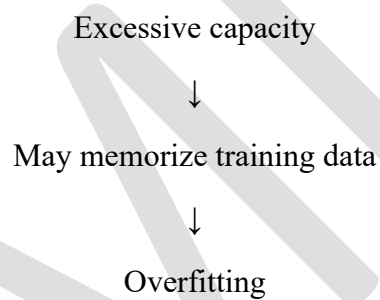
Low capacity



High capacity

Can represent complex patterns

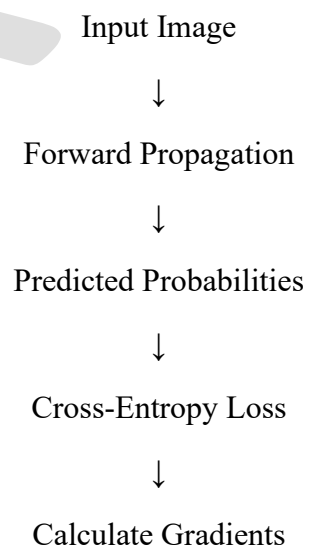
However:

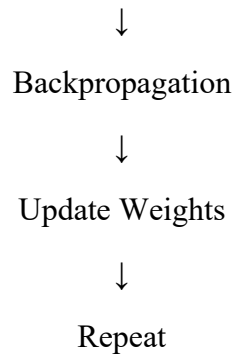


The detailed treatment of overfitting and underfitting comes later in Unit III.

2.23 Optimization Method for Classification

The training process is:





The general parameter update remains:

$$\theta_{new} = \theta_{old} - \eta \nabla_{\theta} L$$

2.24 Adam Optimizer

Adam is a popular gradient-based optimization algorithm.

Example:

```
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",  
metrics=["accuracy"])
```

Adam maintains adaptive estimates based on the gradients and their squared values, allowing different parameters to receive appropriately scaled updates.

It is often used as a practical default optimizer for neural-network training.

2.25 Complete Regression vs Classification Comparison

Feature	Regression	Classification
Output	Continuous value	Class/category
Example	House price	Digit recognition
Output activation	Linear	Sigmoid/Softmax
Common loss	MSE	Cross-entropy
Example output	₹65.4 lakh	Digit 7
Evaluation	MAE, MSE, RMSE	Accuracy, Precision, Recall
Keras output	Dense(1)	Dense(number of classes)

KNMT